

# Java Collections for Beginners (Tier 2)

A practical guide to more specialized collections beyond the basics. We cover collections that solve specific problems: keeping insertion order, sorting items automatically, working as queues or stacks, and defining custom ordering.

---

## What's in Tier 2?

1. `LinkedList`: another way to make a list
2. `LinkedHashMap` and `LinkedHashSet`: collections that remember insertion order
3. `TreeMap` and `TreeSet`: collections that keep items sorted
4. `Queue`: FIFO (first-in, first-out) collections
5. `Deque` and `ArrayDeque`: collections you can use from both ends
6. `PriorityQueue`: queue ordered by priority
7. `Comparable` and `Comparator`: how to define ordering
8. The `Collections` utility class

Let us go through them one by one.

---

## 1. `LinkedList` : A Different Kind of List

`LinkedList` is another implementation of the `List` interface. It works differently inside compared to `ArrayList`.

### How it works inside

`ArrayList` stores items in an array. When you add an item, it goes into the next slot in the array.

`LinkedList` stores items as nodes that point to each other. Each node holds a value, a pointer to the next node, and a pointer to the previous node.

This different structure means different speed for different operations.

### Speed comparison

| Operation     | <code>ArrayList</code> | <code>LinkedList</code> |
|---------------|------------------------|-------------------------|
| Add to end    | Fast                   | Fast                    |
| Add to start  | Slow                   | Fast                    |
| Add in middle | Slow                   | Slow                    |

| Operation           | ArrayList | LinkedList |
|---------------------|-----------|------------|
| Get by index        | Fast      | Slow       |
| Remove from start   | Slow      | Fast       |
| Remove from end     | Fast      | Fast       |
| Iterate through all | Fast      | Fast       |

For most real-world cases, `ArrayList` is faster overall.

### Creating a LinkedList

```
java
import java.util.LinkedList;
import java.util.List;

List<String> list = new LinkedList<>();
list.add("Alice");
list.add("Bob");
list.add("Carol");
```

It has all the same methods as `ArrayList` because both implement the `List` interface.

### Extra methods unique to LinkedList

`LinkedList` also implements the `Deque` interface, so it has extra methods for working at both ends.

```
java
LinkedList<String> list = new LinkedList<>();

list.addFirst("first");
list.addLast("last");
list.addFirst("very first");

list.getFirst();    // "very first"
list.getLast();     // "last"

list.removeFirst();
list.removeLast();
```

### When to use LinkedList

The honest answer: rarely. `ArrayList` is the better default in almost every case.

Use `LinkedList` only when:

- You add and remove a lot at the start of the list (and not by index)
- You need it as a `Deque`, but `ArrayDeque` is even better for that

For everyday lists, stick with `ArrayList`.

---

## 2. LinkedHashMap and LinkedHashSet : Keep Insertion Order

A regular `HashMap` and `HashSet` do not keep any order. If you need items to come out in the order you put them in, use `LinkedHashMap` or `LinkedHashSet`.

### LinkedHashMap

Same as `HashMap` but it remembers the order in which you added the entries.

```
java
import java.util.LinkedHashMap;
import java.util.Map;

Map<String, Integer> map = new LinkedHashMap<>();
map.put("three", 3);
map.put("one", 1);
map.put("two", 2);

map.forEach((k, v) -> System.out.println(k + "=" + v));
// three=3
// one=1
// two=2
```

The order is what you put in, not sorted.

### LinkedHashSet

Same as `HashSet` but it keeps insertion order.

```
java
```

```
import java.util.LinkedHashSet;
import java.util.Set;

Set<String> set = new LinkedHashSet<>();
set.add("apple");
set.add("cherry");
set.add("banana");

set.forEach(System.out::println);
// apple
// cherry
// banana
```

## When to use these

When you want the speed of a hash-based collection but also need to remember the order.

Common use cases:

- Showing config options in the order they were defined
- Caching where you want to know the access order
- Removing duplicates from a list while keeping the original order

## Removing duplicates while keeping order

This is a classic interview question.

```
java

List<String> withDuplicates = List.of("apple", "banana", "apple", "cherry", "banana")

Set<String> unique = new LinkedHashSet<>(withDuplicates);
List<String> noDuplicates = new ArrayList<>(unique);

System.out.println(noDuplicates);
// [apple, banana, cherry]
```

If you used a regular `HashSet`, the order would be unpredictable.

## Cost of preserving order

`LinkedHashMap` and `LinkedHashSet` use a tiny bit more memory than `HashMap` and `HashSet` because they store extra pointers. The performance difference is small in most cases.

---

### 3. TreeMap and TreeSet : Sorted Order

`TreeMap` and `TreeSet` keep their items in sorted order automatically. You do not have to sort them yourself.

#### TreeSet

```
java

import java.util.TreeSet;
import java.util.Set;

Set<Integer> set = new TreeSet<>();
set.add(5);
set.add(1);
set.add(3);
set.add(2);
set.add(4);

System.out.println(set);    // [1, 2, 3, 4, 5]
```

Notice the order is sorted, not insertion order.

#### TreeMap

```
java

import java.util.TreeMap;
import java.util.Map;

Map<String, Integer> map = new TreeMap<>();
map.put("banana", 2);
map.put("apple", 1);
map.put("cherry", 3);

map.forEach((k, v) -> System.out.println(k + "=" + v));
// apple=1
// banana=2
// cherry=3
```

Keys are kept in sorted order.

#### Useful methods on TreeSet

These methods take advantage of the sorted order.

```
java
```

```

TreeSet<Integer> set = new TreeSet<>(Set.of(1, 3, 5, 7, 9));

set.first();           // 1 (smallest)
set.last();            // 9 (largest)

set.floor(4);          // 3 (largest item less than or equal to 4)
set.ceiling(4);        // 5 (smallest item greater than or equal to 4)
set.lower(5);          // 3 (largest item strictly less than 5)
set.higher(5);         // 7 (smallest item strictly greater than 5)

set.pollFirst();       // 1, removes it
set.pollLast();        // 9, removes it

```

## Range views on TreeSet

```

java

TreeSet<Integer> set = new TreeSet<>(Set.of(1, 3, 5, 7, 9));

set.headSet(5);        // [1, 3] (less than 5)
set.tailSet(5);        // [5, 7, 9] (greater than or equal to 5)
set.subSet(3, 8);      // [3, 5, 7] (3 inclusive to 8 exclusive)

```

## Useful methods on TreeMap

Similar methods work on TreeMap, but on keys.

```

java

TreeMap<String, Integer> map = new TreeMap<>();
map.put("apple", 1);
map.put("banana", 2);
map.put("cherry", 3);
map.put("date", 4);

map.firstKey();         // "apple"
map.lastKey();          // "date"

map.floorKey("c");       // "banana"
map.ceilingKey("c");     // "cherry"

map.firstEntry();        // apple=1
map.lastEntry();         // date=4

map.headMap("cherry");   // {apple=1, banana=2}
map.tailMap("cherry");   // {cherry=3, date=4}

```

How does it know how to sort?

For built-in types like `String`, `Integer`, and `Double`, they already know how to sort themselves.

For your own classes, the class must implement `Comparable`, OR you must provide a `Comparator` when creating the `TreeMap` or `TreeSet`. We will cover both shortly.

When to use `TreeMap` and `TreeSet`

Use these when:

- You need items in sorted order
- You need range queries (find all items between X and Y)
- You need to easily get the smallest or largest item
- You frequently need the floor or ceiling of a value

For just fast lookups without sorting, `HashMap` and `HashSet` are faster.

Speed comparison

| Operation      | HashMap                | TreeMap            |
|----------------|------------------------|--------------------|
| put            | Very fast (constant)   | Fast (logarithmic) |
| get            | Very fast (constant)   | Fast (logarithmic) |
| Iterate sorted | Slow (must sort first) | Free               |
| Range queries  | Slow                   | Fast               |

If you need sorting, `TreeMap` saves you the cost of sorting yourself every time.

4. Queue : FIFO Collections

A Queue is a collection where you add at one end and remove from the other. Like a line at a store: first in, first out.

Basic usage

```
java
```

```

import java.util.Queue;
import java.util.LinkedList;

Queue<String> queue = new LinkedList<>();

queue.offer("Alice");    // add to back
queue.offer("Bob");
queue.offer("Carol");

queue.peek();            // "Alice" (look at front, do not remove)
queue.poll();            // "Alice" (remove from front)
queue.poll();            // "Bob"

queue.isEmpty();         // false (Carol still there)

```

## Two flavors of methods

Queue methods come in pairs. One throws exceptions, one returns null or false.

| Action            | Throwing version       | Non-throwing version  |
|-------------------|------------------------|-----------------------|
| Add to back       | <code>add(e)</code>    | <code>offer(e)</code> |
| Remove from front | <code>remove()</code>  | <code>poll()</code>   |
| Look at front     | <code>element()</code> | <code>peek()</code>   |

The throwing versions throw an exception when the queue is empty (or full, depending on the implementation). The non-throwing versions return null or false.

For most code, prefer `offer`, `poll`, and `peek`. They are safer and easier to handle.

## Common Queue implementations

- `LinkedList`: simple, but slower than `ArrayDeque`
- `ArrayDeque`: faster, recommended
- `PriorityQueue`: elements ordered by priority

## Use case: processing tasks one at a time

```
java
```

```
Queue<String> tasks = new ArrayDeque<>();
tasks.offer("download");
tasks.offer("process");
tasks.offer("upload");

while (!tasks.isEmpty()) {
    String task = tasks.poll();
    System.out.println("Working on: " + task);
}
// Working on: download
// Working on: process
// Working on: upload
```

---

## 5. Deque and ArrayDeque : Both Ends

Deque (pronounced "deck") is a "double-ended queue." You can add and remove from both ends.

### Why use Deque?

You can use a Deque as:

- A queue (FIFO): add at one end, remove at the other
- A stack (LIFO): add and remove from the same end
- Anything in between

### ArrayDeque

The most common Deque implementation. It is faster than LinkedList for queue and stack use cases.

```
java
```

```
import java.util.ArrayDeque;
import java.util.Deque;

Deque<Integer> deque = new ArrayDeque<>();

deque.addFirst(1);    // [1]
deque.addLast(2);     // [1, 2]
deque.addFirst(0);    // [0, 1, 2]
deque.addLast(3);     // [0, 1, 2, 3]

deque.peekFirst();    // 0 (look at front)
deque.peekLast();     // 3 (look at back)

deque.pollFirst();    // 0 (remove from front)
deque.pollLast();     // 3 (remove from back)

System.out.println(deque);    // [1, 2]
```

## Using Deque as a Stack

A stack is a "Last In First Out" collection. Like a stack of plates: you put new ones on top and take from the top.

Deque has `push`, `pop`, and `peek` methods for stack use.

```
java

Deque<Integer> stack = new ArrayDeque<>();

stack.push(1);    // [1]
stack.push(2);    // [2, 1]
stack.push(3);    // [3, 2, 1]

stack.peek();     // 3 (top of stack)
stack.pop();      // 3, removes
stack.pop();      // 2, removes

System.out.println(stack);    // [1]
```

`push` adds to the front. `pop` and `peek` look at the front. So the most recent item is at the top.

## Why not use the Stack class?

Java has an old class called `Stack`. Do not use it. It is based on the older `Vector` class and has performance and design problems.

For stack behavior, always use `Deque` with `ArrayDeque`. This is the modern, recommended approach.

### When to use ArrayDeque

- As a queue (FIFO)
- As a stack (LIFO)
- When you need fast add and remove at both ends
- For any general-purpose double-ended needs

ArrayDeque is one of the most useful collections in Java but many beginners do not know about it.

---

## 6. PriorityQueue: Order by Priority

A PriorityQueue removes items in priority order, not insertion order. By default, smaller items come out first.

### Basic example

```
java
import java.util.PriorityQueue;

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(5);
pq.offer(1);
pq.offer(3);
pq.offer(2);
pq.offer(4);

while (!pq.isEmpty()) {
    System.out.print(pq.poll() + " ");
}
// 1 2 3 4 5
```

Items always come out smallest first.

### Custom priority

You can pass a Comparator to define your own priority.

```
java
```

```
import java.util.Comparator;

// Largest first
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

maxHeap.offer(5);
maxHeap.offer(1);
maxHeap.offer(3);

while (!maxHeap.isEmpty()) {
    System.out.print(maxHeap.poll() + " ");
}
// 5 3 1
```

## Practical use case

```
java

record Task(String name, int priority) {}

PriorityQueue<Task> tasks = new PriorityQueue<>(
    Comparator.comparingInt(Task::priority)
);

tasks.offer(new Task("clean code", 3));
tasks.offer(new Task("fix critical bug", 1));
tasks.offer(new Task("write docs", 2));

while (!tasks.isEmpty()) {
    Task t = tasks.poll();
    System.out.println("Doing: " + t.name());
}
// Doing: fix critical bug
// Doing: write docs
// Doing: clean code
```

## Important: only the front is sorted

PriorityQueue does not sort all items in order. It only guarantees that `peek` and `poll` give you the smallest item. If you iterate through the whole thing, the order is not sorted.

```
java
```

```
PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 3, 2, 4));

for (Integer n : pq) {
    System.out.print(n + " ");
}
// might print: 1 2 3 5 4 (NOT fully sorted)
```

To get sorted output, keep calling `poll`.

### Common use cases

- Top K problems (find top 5 highest paid employees)
- Task scheduling by priority
- Dijkstra's shortest path algorithm
- Merge K sorted lists
- Median finding (with two PriorityQueues)

These come up a lot in interviews.

---

## 7. Comparable and Comparator : Custom Ordering

When you want to sort your own classes, or sort built-in types in a custom way, you use Comparable or Comparator.

### Comparable : natural ordering

A class implements Comparable to define how its instances should be compared.

```
java
```

```

class Person implements Comparable<Person> {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public int getAge() { return age; }

    @Override
    public int compareTo(Person other) {
        return Integer.compare(this.age, other.age);
    }
}

```

The `compareTo` method returns:

- A negative number if `this` is less than `other`
- Zero if they are equal
- A positive number if `this` is greater than `other`

`Integer.compare(a, b)` is a safe way to do this. Avoid `a - b` because it can overflow with very large or very small numbers.

Now you can sort lists of Person:

```

java

List<Person> people = new ArrayList<>(List.of(
    new Person("Carol", 30),
    new Person("Alice", 25),
    new Person("Bob", 28)
));

Collections.sort(people);

// Result: Alice(25), Bob(28), Carol(30)

```

## Comparator: external ordering

A Comparator is a separate object that defines how to compare two things. More flexible because you can have multiple comparators for the same class.

```
java
```

```
Comparator<Person> byAge = (a, b) -> Integer.compare(a.getAge(), b.getAge());
Comparator<Person> byName = (a, b) -> a.getName().compareTo(b.getName());

people.sort(byAge);    // sort by age
people.sort(byName);   // sort by name
```

## Modern Comparator with helper methods

The `Comparator` class has helper methods that make this much cleaner.

```
java
```

```
Comparator<Person> byAge = Comparator.comparingInt(Person::getAge);
Comparator<Person> byName = Comparator.comparing(Person::getName);

people.sort(byAge);
people.sort(byName);
```

## Reversing order

```
java
```

```
people.sort(Comparator.comparingInt(Person::getAge).reversed());
// oldest first
```

## Multiple sort keys

Sort by age, then by name if ages are equal.

```
java
```

```
people.sort(
    Comparator.comparingInt(Person::getAge)
               .thenComparing(Person::getName)
);
```

You can chain `thenComparing` as many times as you need.

## Handling nulls

Use `nullsFirst` or `nullsLast` to handle null values.

```
java
```

```
Comparator<Person> safe = Comparator.nullsFirst(  
    Comparator.comparing(Person::getName)  
);
```

## Comparable vs Comparator: which to use

Use Comparable when:

- There is one obvious "natural" way to compare your objects
- You own the class and can modify it

Use Comparator when:

- You need multiple ways to sort
- You cannot modify the class (it is from a library)
- The sort order depends on context

In modern Java code, Comparator is used much more often than Comparable.

---

## 8. The Collections Utility Class

`Collections` (with an "s") is a class with many static helper methods. Do not confuse it with `Collection` (singular, the interface).

### Sorting

```
java  
  
List<Integer> nums = new ArrayList<>(List.of(3, 1, 4, 1, 5, 9, 2, 6));  
  
Collections.sort(nums);  
// [1, 1, 2, 3, 4, 5, 6, 9]  
  
Collections.sort(nums, Comparator.reverseOrder());  
// [9, 6, 5, 4, 3, 2, 1, 1]
```

You can also use the list's own `sort` method, which is often cleaner.

```
java  
  
nums.sort(null); // natural order  
nums.sort(Comparator.reverseOrder()); // reverse
```

## Reversing

```
java

List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5));
Collections.reverse(nums);
// [5, 4, 3, 2, 1]
```

## Shuffling

```
java

Collections.shuffle(nums);
// random order each time
```

## Min and Max

```
java

List<Integer> nums = List.of(3, 1, 4, 1, 5, 9, 2, 6);

Collections.min(nums);    // 1
Collections.max(nums);    // 9

// With a comparator
Collections.max(nums, Comparator.reverseOrder());    // 1
```

## Frequency

How many times an item appears in a collection.

```
java

List<String> words = List.of("apple", "banana", "apple", "cherry", "apple");
Collections.frequency(words, "apple");    // 3
```

## Empty and singleton collections

```
java

List<Integer> empty = Collections.emptyList();
Set<Integer> emptySet = Collections.emptySet();
Map<String, Integer> emptyMap = Collections.emptyMap();

List<Integer> single = Collections.singletonList(42);
Set<String> singleSet = Collections.singleton("Alice");
```

These are immutable. Useful as default return values.

Unmodifiable wrappers

```
java
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));
List<Integer> readOnly = Collections.unmodifiableList(nums);

readOnly.add(4);    // crash: UnsupportedOperationException
```

The wrapper does not allow modification. Useful for protecting your data when returning from methods.

Filling

```
java
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5));
Collections.fill(nums, 0);
// [0, 0, 0, 0, 0]
```

Useful methods at a glance

| Method                  | What it does                   |
|-------------------------|--------------------------------|
| sort(list)              | Sort in natural order          |
| sort(list, comparator)  | Sort with custom order         |
| reverse(list)           | Reverse the order              |
| shuffle(list)           | Random order                   |
| min(coll)/max(coll)     | Smallest or largest element    |
| frequency(coll, item)   | Count occurrences              |
| emptyList(), emptySet() | Immutable empty collections    |
| singletonList(item)     | Immutable list of one item     |
| unmodifiableList(list)  | Read-only wrapper              |
| fill(list, value)       | Replace all items with a value |
| disjoint(c1, c2)        | True if no common elements     |
|                         |                                |
|                         |                                |

## Putting It All Together

Real examples that combine these collections.

### Example 1: Top 3 highest paid employees

```
java

record Employee(String name, double salary) {}

List<Employee> employees = List.of(
    new Employee("Alice", 90000),
    new Employee("Bob", 75000),
    new Employee("Carol", 85000),
    new Employee("Dan", 100000),
    new Employee("Eve", 60000)
);

PriorityQueue<Employee> top3 = new PriorityQueue<>(
    Comparator.comparingDouble(Employee::salary)
);

for (Employee e : employees) {
    top3.offer(e);
    if (top3.size() > 3) {
        top3.poll(); // remove smallest
    }
}

// top3 now has the 3 highest paid (in unsorted internal order)
```

### Example 2: Word frequency in sorted order

```
java
```

```
String text = "the quick brown fox jumps over the lazy dog the fox";
String[] words = text.split(" ");

TreeMap<String, Integer> counts = new TreeMap<>();
for (String word : words) {
    counts.merge(word, 1, Integer::sum);
}

counts.forEach((word, count) ->
    System.out.println(word + ": " + count)
);
// brown: 1
// dog: 1
// fox: 2
// jumps: 1
// lazy: 1
// over: 1
// quick: 1
// the: 3
```

### Example 3: Browser history with back and forward

```
java
```

```
Deque<String> back = new ArrayDeque<>();
Deque<String> forward = new ArrayDeque<>();

String currentPage = "home";

// Visit a page
void visit(String page) {
    back.push(currentPage);
    currentPage = page;
    forward.clear();
}

// Go back
void goBack() {
    if (!back.isEmpty()) {
        forward.push(currentPage);
        currentPage = back.pop();
    }
}

// Go forward
void goForward() {
    if (!forward.isEmpty()) {
        back.push(currentPage);
        currentPage = forward.pop();
    }
}
```

#### Example 4: Sort employees by multiple criteria

```
java
```

```

List<Employee> employees = new ArrayList<> (List.of(
    new Employee("Alice", "Engineering", 90000),
    new Employee("Bob", "Sales", 75000),
    new Employee("Carol", "Engineering", 85000),
    new Employee("Dan", "Sales", 95000)
));

employees.sort(
    Comparator.comparing(Employee::department)
                .thenComparing(Comparator.comparingDouble(Employee::salary).reversed())
);

// Result: Engineering Alice 90k, Engineering Carol 85k,
// Sales Dan 95k, Sales Bob 75k

```

### Example 5: Get unique items in original order

```

java

List<String> items = List.of("apple", "banana", "apple", "cherry", "banana", "date")

List<String> unique = new ArrayList<> (new LinkedHashSet<> (items));
System.out.println(unique);
// [apple, banana, cherry, date]

```

### Example 6: Range query for events

```

java

TreeMap<LocalDate, String> events = new TreeMap<> ();
events.put(LocalDate.of(2026, 1, 15), "New Year Party");
events.put(LocalDate.of(2026, 3, 10), "Tech Conf");
events.put(LocalDate.of(2026, 6, 1), "Summer Picnic");
events.put(LocalDate.of(2026, 9, 5), "Hackathon");

// Get all events between two dates
NavigableMap<LocalDate, String> q1Events = events.subMap(
    LocalDate.of(2026, 1, 1), true,
    LocalDate.of(2026, 3, 31), true
);

q1Events.forEach((date, event) ->
    System.out.println(date + ": " + event)
);

```

## Example 7: BFS using a queue

A typical breadth-first traversal pattern for trees and graphs.

```
java

Queue<Node> queue = new ArrayDeque<>();
queue.offer(root);

while (!queue.isEmpty()) {
    Node current = queue.poll();
    System.out.println(current.value);

    for (Node child : current.children) {
        queue.offer(child);
    }
}
```

---

## Common Mistakes

### Mistake 1: Confusing Collections (utility) with Collection (interface)

```
java

// Collection (singular): the interface for List, Set, Queue
Collection<String> coll = new ArrayList<>();

// Collections (plural): the utility class
Collections.sort(list);
```

Easy to mix up. Watch for the "s".

### Mistake 2: Using TreeSet or TreeMap with non-Comparable types

```
java

class Person {
    String name;
    int age;
}

Set<Person> set = new TreeSet<>();
set.add(new Person("Alice", 25)); // ClassCastException
```

`TreeSet` needs to compare items. Either implement `Comparable` on your class, or pass a `Comparator`.

```
java
Set<Person> set = new TreeSet<>(Comparator.comparing(p -> p.name));
```

### Mistake 3: Putting null in TreeSet or TreeMap

```
java
TreeSet<Integer> set = new TreeSet<>();
set.add(null);    // NullPointerException
```

Sorted collections cannot compare null values. They reject them.

### Mistake 4: Using Stack class

```
java
Stack<Integer> stack = new Stack<>();    // legacy, avoid
```

Use ArrayDeque instead.

```
java
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.pop();
```

### Mistake 5: Using LinkedList when ArrayList would do

```
java
List<Integer> list = new LinkedList<>();    // usually slower
```

Default to ArrayList. Only use LinkedList when you have a specific reason.

### Mistake 6: Iterating a PriorityQueue and expecting sorted order

```
java
PriorityQueue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 3));

for (Integer n : pq) {
    System.out.print(n);    // not sorted!
}
```

Only `poll()` gives sorted order. Iteration does not.

### Mistake 7: Modifying a sub-view of a TreeSet

```
java

TreeSet<Integer> set = new TreeSet<>(Set.of(1, 3, 5, 7, 9));
SortedSet<Integer> head = set.headSet(5);
head.add(10);    // IllegalArgumentException, 10 is not less than 5
```

The view `headSet(5)` only lets you put values that fit its range (less than 5). Adding 10 to it would break that rule.

### Mistake 8: Using `a - b` in compareTo

```
java

@Override
public int compareTo(Person other) {
    return this.age - other.age;    // can overflow!
}
```

If ages can be very large or negative, this can wrap around and give wrong results. Always use `Integer.compare`.

```
java

return Integer.compare(this.age, other.age);
```

### Mistake 9: Forgetting that some operations on TreeMap views are slow

```
java

TreeMap<String, Integer> map = new TreeMap<>();
// ...
map.headMap("z").size();    // counts all matching entries, takes time
```

The view is cheap to create, but operations on it still scan the entries.

### Mistake 10: Using compareTo for equality

```
java

Person p1 = new Person("Alice", 25);
Person p2 = new Person("Bob", 25);

p1.compareTo(p2) == 0;    // true, but they are not equal!
```

`compareTo` returning zero means "same priority for sorting." It does NOT mean "equal." Use `equals` for equality and `compareTo` for ordering. They should be consistent but they answer different questions.

---

## Quick Cheat Sheet

### Choosing a Collection

| You need...                            | Use                        |
|----------------------------------------|----------------------------|
| Ordered list, fast index access        | <code>ArrayList</code>     |
| Unique items, no order needed          | <code>HashSet</code>       |
| Key-value lookup, no order             | <code>HashMap</code>       |
| Key-value lookup, keep insertion order | <code>LinkedHashMap</code> |
| Unique items, keep insertion order     | <code>LinkedHashSet</code> |
| Key-value lookup, sorted by key        | <code>TreeMap</code>       |
| Unique items, sorted                   | <code>TreeSet</code>       |
| Queue (FIFO)                           | <code>ArrayDeque</code>    |
| Stack (LIFO)                           | <code>ArrayDeque</code>    |
| Priority-based processing              | <code>PriorityQueue</code> |
| Frequent add/remove at start of list   | <code>LinkedList</code>    |

### Comparator helpers

```
java
```

```
Comparator.naturalOrder()  
Comparator.reverseOrder()  
Comparator.comparing(Person::getName)  
Comparator.comparingInt(Person::getAge)  
Comparator.comparing(Person::getName).reversed()  
Comparator.comparing(Person::getAge).thenComparing(Person::getName)  
Comparator.nullsFirst(Comparator.comparing(Person::getName))
```

## Common Collections methods

java

```
Collections.sort(list);
Collections.sort(list, comparator);
Collections.reverse(list);
Collections.shuffle(list);
Collections.min(coll);
Collections.max(coll);
Collections.frequency(coll, item);
Collections.emptyList();
Collections.unmodifiableList(list);
```

## Queue/Deque method cheat

| Operation         | Throws on empty                                | Returns null on empty                       |
|-------------------|------------------------------------------------|---------------------------------------------|
| Add to back       | <code>add</code> / <code>addLast</code>        | <code>offer</code> / <code>offerLast</code> |
| Remove from front | <code>remove</code> / <code>removeFirst</code> | <code>poll</code> / <code>pollFirst</code>  |
| Look at front     | <code>element</code> / <code>getFirst</code>   | <code>peek</code> / <code>peekFirst</code>  |
| Add to front      | <code>addFirst</code>                          | <code>offerFirst</code>                     |
| Remove from back  | <code>removeLast</code>                        | <code>pollLast</code>                       |
| Look at back      | <code>getLast</code>                           | <code>peekLast</code>                       |

## Final Tips

1. For most lists, use `ArrayList`. For most maps, use `HashMap`. For most sets, use `HashSet`. Reach for the specialized ones only when you have a specific need.
2. If you need insertion order, use `LinkedHashMap` or `LinkedHashSet`. If you need sorted order, use `TreeMap` or `TreeSet`.
3. For stack and queue behavior, always use `ArrayDeque`. Never use the old `Stack` class.
4. `PriorityQueue` is great for top-K problems and task scheduling. Remember that iteration is not in sorted order.
5. Use `Comparator.comparing` and friends. They are much cleaner than writing comparator lambdas by hand.

6. Use `Integer.compare(a, b)` instead of `a - b` in your compare methods. It avoids overflow.
7. Sub-views from `TreeSet` and `TreeMap` (like `headSet`, `subMap`) are live views. Changes to one affect the other.
8. `equals` and `compareTo` should agree. If two objects are equal, `compareTo` should return zero. If `compareTo` returns zero, they should usually be equal.
9. `Collections.unmodifiableList(...)` and similar wrappers protect your data when returning from methods. The original is still mutable, only the wrapper is read-only.
10. Practice common patterns: top-K with `PriorityQueue`, frequency count with `HashMap`, BFS with `Queue`, removing duplicates while keeping order with `LinkedHashSet`. These come up everywhere.

You now know most of the collections you will need in everyday Java programming. The next tier covers concurrent collections (for multi-threaded code), specialized collections like `EnumMap` and `IdentityHashMap`, and deeper performance topics.